

This diagram illustrates the architecture of a stock trading system, organized into several layers and components.

Interfaces

- Driver** (Interface):
 - + Tick(): error
 - + Snapshot(): (IStockQuote, error)
 - + ApplyBuy(ticker string, quantity uint64): error
 - + ApplySell(ticker string, quantity uint64): error
- InventoryStore** (Interface):
 - + SaveQuantity(ctx context.Context, ticker string, quantity uint64): error
- Publisher** (Interface):
 - + StoreQuotes(ctx context.Context, quotes []IStockQuote): error
 - + PublishQuotesUpdated(ctx context.Context, quotes []IStockQuote): error
 - + PublishCompanyBuyResult(ctx context.Context, result CompanyBuyResult): error
 - + PublishCompanySellResult(ctx context.Context, result CompanySellResult): error
- CompanyTradeHandler** (Interface):
 - + HandleCompanyBuy(ctx context.Context, event CompanyBuyRequested): (CompanyBuyResult, error)
 - + HandleCompanySell(ctx context.Context, event CompanySellRequested): (CompanySellResult, error)

Config

- Config**:
 - + HTTPAddr: string
 - + RedisAddr: string
 - + RedisPassword: string
 - + RedisDB: int
 - + InventoryDB: InventoryDBConfig
 - + MarketDriverConfig: string
 - + TickInterval: time.Duration
 - + ConsumerGroup: string
 - + ConsumerName: string
- InventoryDBConfig**:
 - + Host: string
 - + Port: int
 - + Name: string
 - + User: string
 - + Password: string
 - + SSLMode: string
 - + Enabled(): bool

Market Engine

- Service**:
 - driver: Driver
 - publisher: Publisher
 - inventory: InventoryStore
 - tickInterval: time.Duration
 - log: *log.Logger
 - + SetInventoryStore(store InventoryStore)
 - + StartTickLoop(ctx context.Context)
 - + TickOnce(ctx context.Context): error
 - + Snapshot(): (IStockQuote, error)
 - + HandleCompanyBuy(ctx context.Context, event CompanyBuyRequested): (CompanyBuyResult, error)
 - + HandleCompanySell(ctx context.Context, event CompanySellRequested): (CompanySellResult, error)
- PackageDriver**:
 - + Tick(): error
 - + Snapshot(): (IStockQuote, error)
 - + ApplyBuy(ticker string, quantity uint64): error
 - + ApplySell(ticker string, quantity uint64): error
 - + SetAvailableQuantity(ticker string, quantity uint64): error

Domain Models

- StockQuote**:
 - + Ticker: string
 - + Price: float64
 - + AvailableQuantity: uint64
 - + Volatility: float64
 - + UpdatedAtUnix: int64
- CompanyBuyRequested**:
 - + EventID: string
 - + TradeID: string
 - + UserID: string
 - + Ticker: string
 - + Quantity: uint64
 - + Price: float64
- CompanyBuyResult**:
 - + EventID: string
 - + TradeID: string
 - + Ticker: string
 - + Quantity: uint64
 - + Accepted: bool
 - + ActualPrice: float64
 - + RemainingAvailableQuantity: uint64
 - + Reason: *string (optional)
- CompanySellRequested**:
 - + EventID: string
 - + TradeID: string
 - + UserID: string
 - + Ticker: string
 - + Quantity: uint64
 - + Price: float64
- CompanySellResult**:
 - + EventID: string
 - + TradeID: string
 - + UserID: string
 - + Ticker: string
 - + Quantity: uint64
 - + Accepted: bool
 - + ActualPrice: float64
 - + RemainingAvailableQuantity: uint64
 - + Reason: *string (optional)

Redis Layer

- redis.Client**:
 - addr: string
 - password: string
 - db: int
 - timeout: time.Duration
 - + Do(ctx context.Context, args ...any): (any, error)
 - + dial(ctx context.Context): (net.Conn, error)
- redis.Publisher**:
 - client: *Client
 - + StoreQuotes(ctx context.Context, quotes []IStockQuote): error
 - + PublishQuotesUpdated(ctx context.Context, quotes []IStockQuote): error
 - + PublishCompanyBuyResult(ctx context.Context, result CompanyBuyResult): error
 - + PublishCompanySellResult(ctx context.Context, result CompanySellResult): error
 - + LoadQuotes(ctx context.Context): ([]IStockQuote, error)
 - + LoadQuote(ctx context.Context, ticker string): (IStockQuote, bool, error)
- redis.Consumer**:
 - client: *Client
 - handler: CompanyTradeHandler
 - group: string
 - consumerName: string
 - log: *log.Logger
 - + Start(ctx context.Context)
 - consume(ctx, stream string, handler func(context.Context, string, StreamMessage))
- StreamMessage**:
 - + ID: string
 - + Fields: map[string]string
- QuotesUpdatedEvent**:
 - + Type: string
 - + Quotes: []IStockQuote

Persistence

- inventorypg.Repository**:
 - db: *sql.DB
 - log: *log.Logger
 - + Close(): error
 - + LoadQuantities(ctx context.Context): (map[string]uint64, error)
 - + SaveQuantity(ctx context.Context, ticker string, quantity uint64): error

HTTP Layer

- inventoryHandler**:
 - ctx: context.Context
 - service: inventoryService
 - redis: redisCommander
 - + handleDecrement(w http.ResponseWriter, r *http.Request)
 - + handleIncrement(w http.ResponseWriter, r *http.Request)
- inventoryRequest**:
 - + Ticker: string
 - + Quantity: string
 - + IdempotencyKey: string
- inventoryResponse**:
 - + Ticker: string
 - + Remaining: string
 - + IdempotencyKey: string
 - + Applied: bool
- storedOutcome**:
 - + StatusCode: int
 - + Response: *inventoryResponse
 - + Error: *errorResponse
 - + DuplicateApplied: bool

Text is not SVG - cannot display

Text is not SVG - cannot display